Automated Software Engineering Course Project 2026/2027

(2026/09/29: versão 1.0 do enunciado)

1. Introduction

In this project, you will design, develop, deploy, and operate **MetroIsJustABus**, a distributed software system supporting an urban bus network and its passenger services. The system is intentionally designed to involve multiple interacting components, stateful operations, concurrency, failures, and cross-component interactions.

Large Language Models (LLMs) and AI-based software engineering tools must be used throughout the software development lifecycle, including requirements engineering, design, implementation, testing, documentation, infrastructure configuration, deployment, and system evolution. Teams are expected to progressively adopt an agentic development process, providing AI agents with the context, tools, constraints, and validation mechanisms required to perform increasingly complex engineering tasks.

The goal of the project is not only to produce a working software system, but to establish a reliable engineering process for developing software with LLMs and AI agents. Teams must critically assess AI-generated artifacts, identify incorrect assumptions and failures, establish appropriate validation mechanisms, and maintain traceability between engineering activities and the resulting software artifacts.

The complexity of MetroIsJustABus is an intentional part of the project. Requirements and design decisions may affect multiple components and propagate across application code, interfaces, tests, infrastructure, and deployment configuration. Teams must therefore address system-level integration, consistency, failure handling, and evolution rather than treating individual components as independent generation tasks.

Teams must design and implement a continuous integration, continuous delivery, and continuous deployment (CI/CDE/CD) process supporting the development and operation of the system. This includes establishing the required development and production infrastructure and automating appropriate integration, testing, quality assurance, artifact creation, deployment, validation, and release activities.

Throughout the project, teams must collect and analyze evidence about their AI-assisted and agentic development process. This includes development and review effort, agent autonomy and human intervention, validation results, defects and rework, and CI/CDE/CD behavior. The resulting evidence should support a critical analysis of where LLMs and AI agents were effective, where they failed or required human intervention, and which engineering practices and controls contributed to successful development and delivery.

2. Objectives

The project is designed to provide experience with the end-to-end software development lifecycle (SDLC), using AI and agentic software engineering practices together with continuous validation and DevOps. By the end of the project, you should:

- Experience the full SDLC, from requirements engineering and system modeling to implementation, deployment, and operation;
- Use LLMs and AI agents to support software engineering activities across the development lifecycle;
- Design and maintain consistent requirements, data models, software architecture, interfaces, implementation, and verification artifacts;
- Apply systematic validation techniques to assess the correctness and quality of AI-generated software and engineering artifacts;
- Design and implement CI/CDE/CD pipelines for automated testing, integration, release, and deployment;
- Automate the deployment and operation of a multi-component system using containerization and infrastructure configuration;
- Collect and analyze evidence about the effectiveness, limitations, autonomy, and human intervention associated with AI-assisted and agentic development;
- Develop autonomy and critical judgment in the use of AI and modern software engineering practices.

3. Project Structure

The project will be completed in teams of 5 students. You will:

- Analyze the project description and derive the system requirements;
- Model and design the system, including its data model, architecture, components, interfaces, and relevant behavioral aspects;
- Develop the system using LLMs and AI agents throughout the software development lifecycle;
- Establish appropriate validation mechanisms for AI-generated software and engineering artifacts;
- Progressively configure and use AI agents to perform increasingly complex software engineering tasks;
- Set up a CI/CDE/CD pipeline to automate integration, testing, quality assurance, release, and deployment;
- Containerize and deploy the system to development and production infrastructure, using an adequate deployment strategy for the system;
- Monitor and validate the behavior of the deployed system;
- Collect metrics and evidence about the development process, including AI/agent performance, human intervention, defects, rework, validation, and pipeline behavior;
- Analyze and critically reflect on the effectiveness and limitations of the AI-assisted and agentic software engineering process.

4. Project Phases

Requirements & Design

Analyze the provided problem and stakeholder information to identify and refine the system requirements. Develop the system models and architecture, including the data model, component architecture, interfaces, and relevant behavioral models. LLMs and AI agents should support the generation, analysis, refinement, and validation of these artifacts.

Implementation

Implement the system using LLMs and AI agents for activities such as code generation, refactoring, debugging, integration, and documentation. AI agents should progressively be provided with the context, tools, and constraints required to perform increasingly complex development tasks.

Testing & Verification

Develop and automate appropriate verification mechanisms, including unit, integration, system, and acceptance testing. Use additional techniques such as static analysis, coverage analysis, mutation testing, and performance or security testing where appropriate. AI-generated artifacts must be independently validated rather than assumed to be correct.

CI/CD Pipeline

Design and implement a CI/CDE/CD pipeline covering integration, automated testing, quality assurance, release management, and deployment. The pipeline should provide automated feedback to developers and AI agents and establish appropriate validation gates before software reaches production.

Deployment & Monitoring

Containerize and deploy the system using development and production infrastructure. Select and implement an appropriate deployment strategy, such as rolling, blue-green, canary, shadow, or A/B deployment. Configure monitoring and observability mechanisms to evaluate the behavior of the deployed system and support the detection and diagnosis of failures.

Measurement & Critical Analysis

Collect quantitative and qualitative evidence throughout the project about the AI-assisted and agentic development process. Analyze development and review effort, agent autonomy and human intervention, defects and rework, verification results, CI/CD behavior, and other relevant metrics. Use this evidence to identify where AI and AI agents were effective, where they failed, and which engineering practices contributed to successful outcomes.

5. Milestones and Deliveries

Important: all members are required to attend all the evaluation periods

Milestone 1 (during class) - Requirements (5%) – October 27th

By this milestone, teams must have analyzed the provided problem and stakeholder information and established an initial specification of the system. At minimum:

- The main functional and non-functional requirements must be identified and documented.
- Relevant ambiguities, inconsistencies, missing information, and assumptions identified during requirements analysis must be documented.
- Requirements must be traceable to the information and constraints presented in the project description.
- An initial decomposition of the system into its main components and responsibilities must be proposed.
- Teams must provide evidence of how LLMs and/or AI agents were used to support requirements analysis, refinement, and validation.
- Teams must be able to explain and justify the resulting requirements and the main decisions made during their analysis.

Milestone 2 (during class) - Code created as part of a CI/CDE/CD Pipeline (5%) – November 17th

By this milestone, teams must have a basic CI/CDE/CD Pipeline in operation. At minimum:

- The project must be automatically built from the repository;
- Automated tests must be executed as part of the pipeline;
- Basic code quality checks must be included;
- The application must be containerized;
- A successful pipeline execution must produce a deployable version of the system;
- The pipeline must deploy the application to the development environment.

Final Delivery (90%) – December 11th

- Source Code and Engineering Artifacts:
  - Include the source code, scripts, configuration files, models, documentation, and other artifacts necessary to build, test, deploy, and operate the software.
  - Include the complete CI/CDE/CD and deployment configuration.
  - The final version of the system must be deployed and operational in the production environment.

- Development Evidence and Metrics:
  - Include the collected evidence and metrics regarding the use of LLMs and AI agents throughout the project.
  - Include sufficient records to support the analysis of agent activity, human intervention, validation, defects, rework, and development effort.

- Final report documenting the project and the engineering process followed throughout its development. The report should include/discuss:
    - Problem analysis and requirements engineering, including how the provided stakeholder information was analyzed, identified ambiguities or conflicts, assumptions made, and the resulting requirements.
    - System design, including the domain and data models, architecture, component responsibilities, interfaces, communication mechanisms, and relevant architectural decisions and trade-offs.
    - Traceability between requirements, design decisions, implementation tasks, verification activities, and the resulting software artifacts.
    - AI-assisted and agentic development process, including the tools and models used, how agents were configured and provided with context, the types of tasks delegated to them, and how the use of agents evolved throughout the project.
    - Testing and verification strategy, including unit, integration, system and acceptance testing, verification of component interactions, static analysis, and other techniques used to validate AI-generated artifacts.
    - CI/CDE/CD process, including pipeline architecture, validation gates, artifact creation and promotion, deployment to development and production environments, and mechanisms for handling unsuccessful builds or deployments.
    - Deployment and operation, including containerization, infrastructure configuration, deployment strategy, configuration and secret management, persistent data, monitoring, logging, and mechanisms used to diagnose operational failures.
    - Development evidence, metrics, and critical analysis, including development effort, agent autonomy and human intervention, iterations and rework, defects and validation failures, testing and CI/CD results, and representative development tasks. The analysis should identify where AI agents were effective, where they failed or required human intervention, and which engineering practices and validation mechanisms were effective in detecting or preventing problems.
    - Team contributions, identifying the main tasks performed by each team member and the associated effort.
- 10-min video demonstration of the final MetroIsJustABus system, showing:
    - The main functionality of the system from the passenger perspective.
    - Relevant interactions between the different components of the system.
    - The behavior of the system in relevant normal and failure scenarios.
    - Evidence that the demonstrated system corresponds to the deployed version submitted for evaluation.
- Self- and Peer-evaluations:
    - You will be given a form to evaluate your performance and that of your team members.

Defenses

The group must sign up for an available time slot for the defense in Inforestudante before the final delivery deadline (the list of available slots will be released before the deadline).

All students must be present, but the defense will be individual, ordered by name.
During the defense, every student must be able to justify the design choices and implementation details of all aspects of the project, regardless of who developed or implemented them. This is a group project and all members are expected to understand the entire system and the development process followed.

6. Evaluation Criteria

The table below summarizes the evaluation criteria, their weight, and what is expected for each component. This is a group-project, but the grades are individually assigned.

Criterion	Weight	Description
Process & Agentic Engineering	30%	Development process, teamwork, version control, traceability, and effective use, configuration, and supervision of LLMs and AI agents throughout the project.
Technical Implementation & Delivery	30%	System functionality and quality, requirements and design consistency, testing and verification, CI/CDE/CD pipeline, infrastructure, deployment strategy, and monitoring
Measurement & Critical Analysis	30%	Collection and analysis of development evidence and metrics, including AI/agent effectiveness, autonomy, human intervention, defects, rework, validation results, and pipeline behavior.
Reporting & Demonstration	10%	Clarity, structure, and quality of the final report, demonstration, and evidence-based conclusions.

Technical Description – MetroIsJustABus

MetroIsJustABus operates an urban bus network composed of lines and stops. A line defines an ordered sequence of stops. The network operates according to published schedules, which define the trips planned for each line and the expected times at which those trips serve each stop. Schedules may vary according to the day of the week or other operational conditions.

Passengers should be able to consult the network, lines, stops, schedules, and scheduled trips. Information presented to passengers should consider known changes to normal operation.

During operation, scheduled trips may be delayed, cancelled, diverted, or otherwise disrupted. Disruptions may affect a complete line, individual stops, particular trips, or periods of operation. Information about disruptions should be reflected in the information provided to passengers.

MetroIsJustABus also supports passenger access to the transport network through different types of tickets. Ticket types may differ in their price, period of validity, conditions of use, or number of permitted journeys. Passengers should be able to acquire tickets and consult the tickets associated with their account.

Tickets must be validated when they are used. MetroIsJustABus must determine whether a ticket is valid for the attempted use and record successful validations. Ticket validity may depend on the type of ticket, its current state, previous validations, and the time at which it is used. Operational staff should be able to obtain information about ticket usage and validation activity.

The system should also support monitoring the occupancy of scheduled trips and their available capacity. Passenger activity, including ticket validations, may affect the occupancy information associated with a trip. Capacity information should be available to operational staff and, where appropriate, to passengers.

Passengers may create accounts and subscribe to information relevant to them. For example, a passenger may be interested in a particular line, stop, or scheduled trip. When something relevant changes, MetroIsJustABus should determine which passengers are affected and notify them. Passengers should be able to control their subscriptions and notification preferences.

Operational staff require mechanisms for maintaining network and schedule information, managing ticket-related information, and registering and managing disruptions. Changes made by operational staff may affect information already being used by passengers and may result in notifications being generated.

The information managed by MetroIsJustABus has different lifecycles. Some information represents relatively stable aspects of the transport network, while schedules, trips,

disruptions, tickets, validations, subscriptions, and notifications change over time. Historical information may be relevant for understanding previous system activity and operation.

Temporary failures of individual parts of MetroIsJustABus should not unnecessarily make the entire system unavailable. For example, a failure of the notification functionality should not prevent passengers from consulting lines, stops, and schedules or using other unrelated functionality. The system should also handle communication failures without causing unintended effects, such as issuing the same ticket, recording the same validation, or sending the same notification multiple times.

Application and Architecture

MetroIsJustABus must be implemented as a distributed, multi-component system. The distributed nature of the system is an intentional part of the project and should expose the development process to integration, interoperability, failures, and cross-component interactions.

The system must include a core information system responsible for the main MetroIsJustABus domain and independently deployable supporting components or services. These components must interact through explicitly defined interfaces rather than being implemented exclusively as modules of a single application.

The architectural decomposition is a design decision. Teams must define and justify the responsibilities and boundaries of the different components and the mechanisms through which they communicate.

At minimum, the solution must include:

- A passenger-facing web application;
- A core backend responsible for the main MetroIsJustABus information and functionality;
- One or more independently deployable supporting backend components or services;
- A notification capability implemented across an independent application boundary;
- Appropriate APIs, messages, or events for communication between components.

A single backend application providing all system functionality through a REST API does not satisfy the distributed system requirement.

Communication and Integration

The system must demonstrate interoperability across application boundaries. The solution must include:

- Synchronous communication through appropriate APIs;
- Asynchronous/event-based communication for appropriate operations;
- Explicitly defined interfaces and data exchanged between components;
- Handling of communication failures, timeouts, and temporary component unavailability;

- Appropriate handling of retries and duplicate messages or operations;
- Appropriate mechanisms for maintaining consistency when an operation involves multiple components.

Teams must justify the selection of communication mechanisms, protocols, and data formats.

Testing and Verification

Testing must address both individual components and their interactions. The solution must include:

- Automated unit tests;
- Automated integration tests;
- Automated system/end-to-end tests;
- Verification of interfaces and interactions between independently deployed components;
- Automated code quality and static analysis;
- Tests covering relevant failure and recovery scenarios;
- Additional verification techniques appropriate to the identified functional and non-functional requirements.

CI/CDE/CD

Teams must implement a CI/CDE/CD pipeline supporting the development and delivery of the complete system. The pipeline must include:

- Automated building and testing;
- Automated quality checks;
- Creation of versioned deployable artifacts;
- Integration and system validation;
- Automated deployment to the development/production environments;
- Appropriate validation gates before production deployment;
- Traceability between source-code versions, generated artifacts, and deployed versions.

Deployment and Infrastructure

Teams are responsible for provisioning and configuring the infrastructure required to develop, deploy, and operate MetroIsJustABus. At minimum, separate development and production environments must be established.

The development environment must support the development and delivery process, including a self-hosted CI/CD runner and the execution of integration and system tests. The production environment must host the production version of MetroIsJustABus and its supporting components.

The setup and configuration of the environments must be reproducible and maintained as part of the project.

The solution must include:

- Containerization of the application components and supporting services;
- Version-controlled deployment and infrastructure configuration;
- Automated deployment through the CI/CDE/CD pipeline;
- Appropriate separation between development and production;
- Selection and implementation of an appropriate deployment strategy;
- Management of configuration, secrets, persistent data, networks, and service dependencies;
- Mechanisms for validating deployments and recovering from unsuccessful deployments.

AI-Assisted and Agentic Engineering

LLMs and AI agents must be used throughout the development of MetroIsJustABus, including activities beyond code generation. Teams are expected to progressively configure and use AI-assisted or agentic workflows for requirements engineering, design, implementation, testing, integration, deployment, and other appropriate software engineering activities.

The use of AI must be integrated with the engineering and validation process. Teams must establish mechanisms for providing agents with appropriate project context and constraints, validating their outputs, and determining when human intervention is required.

The development process must produce sufficient evidence to evaluate the use of LLMs and agents, including the tasks delegated to them, the level of human intervention required, validation results, failures and rework, and their effect on the development and delivery process.